

Implementación de Maquinas de Estados Finitos

1- Dr. Lic. Roberto Federico FARFÁN

farfan.roberto.f@gmail.com

2- Esp. Ing. Luis Mariano CAMPOS

1,2-Profesor de Introducción a los Sistemas Embebidos - **Facultad de Ingeniería – UBA**

1-Profesor de Electrónica, Electrónica Analógica y Digital - **Facultad de Ingeniería - U.N.Sa**

2- **CONICET**

Resumen de la clase:

- 5.1 Temas para la semana 5.
- 5.2 Repaso: Maquina de Estado Finito (MEF).
- 5.3 Diagrama de Estados.
- 5.4 Problemas.
- 5.5 Ciclo de Histéresis con MEF.
- 5.6 Teclado Matricial.

Temas para la semana 5

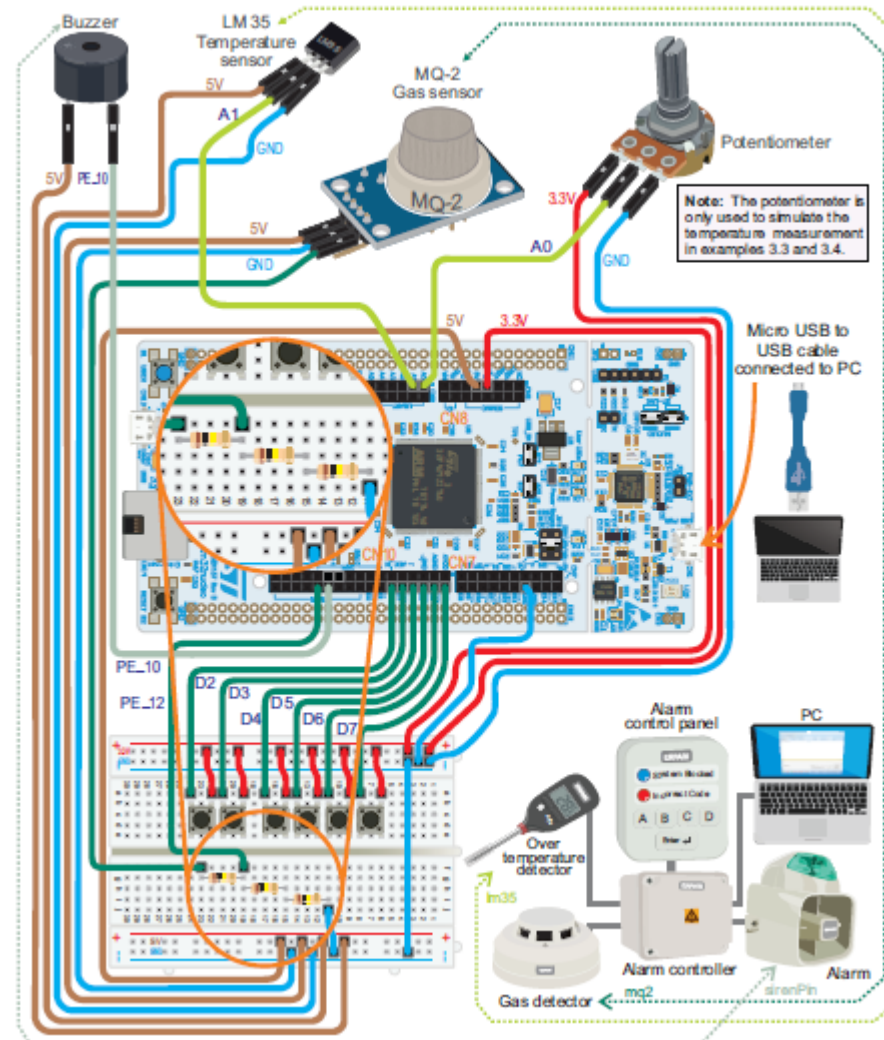


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

3.1 Temas para la semana 3.

4	Contenido de las clases de la semana 4. Teoría: Temporizadores (<u>Timers</u>) y sus registros de control. Interrupciones externas. Concepto de frecuencia. PWM. Práctica: Generación de señales PWM. Implementación de retardos y generación de tiempos periódicos. Uso de botones por interrupción. Interfaz <u>Node-RED</u> y placa NUCLEO-F446RE, haciendo uso de interrupciones externas.	Será informada por el docente a cargo.	1, 2, 3 y 4.
5	Contenido de las clases de la semana 5. Teoría: Organización de programas mediante Máquinas de Estados Finitas (FSM). <u>Modularización</u> aplicada a la programación de sistemas embebidos. Grafo. Práctica: Diseño de lógica de control secuencial con estados y transiciones. Visualización de los estados en <u>Node-RED</u> enviados por la placa NUCLEO-F446RE.	Será informada por el docente a cargo.	1, 2, 3 y 4.

Repaso: MEF

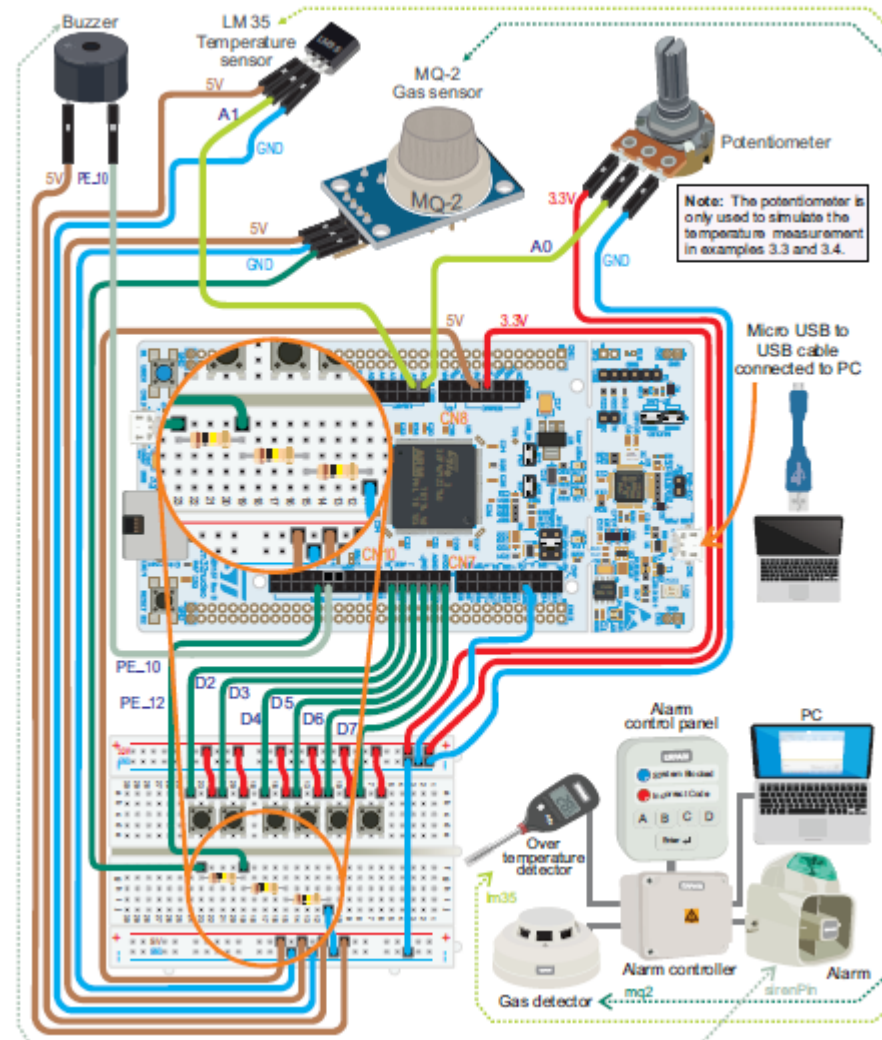
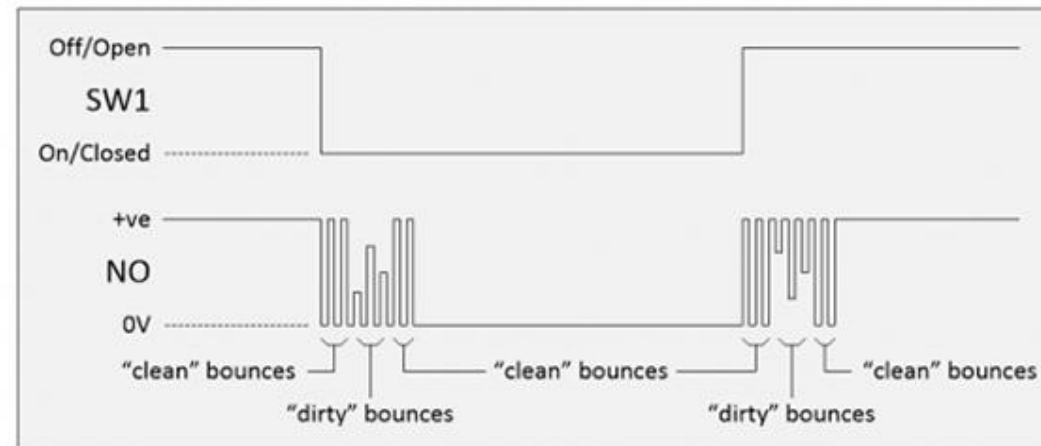
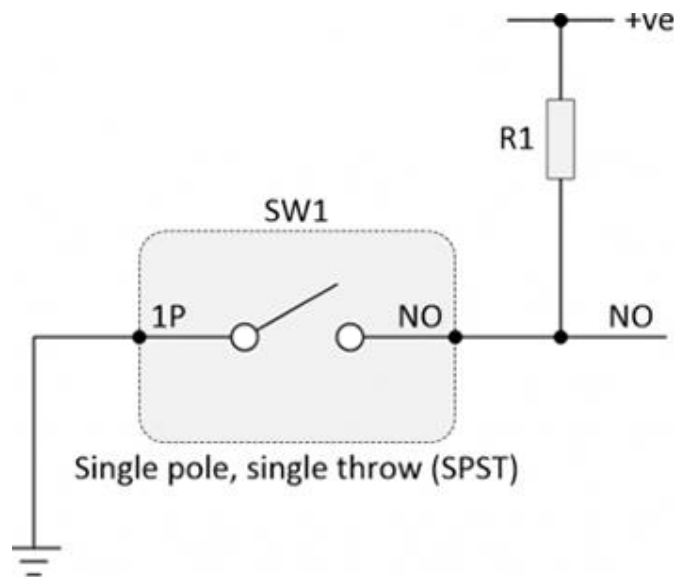


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

5.2. Repaso: MEF.

- Rebotes (Bouncing): las señales eléctricas de los botones mecánicos no son perfectas; presentan ruidos rápidos y oscilaciones al ser presionados o soltados.



5.2. Repaso: MEF.

- Es fundamental implementar una secuencia de "debounce" para evitar falsas detecciones.
- Hay variaciones en la señal por ruido eléctrico, mientras que los rebotes son consecuencia del muelle de la tecla o botón.

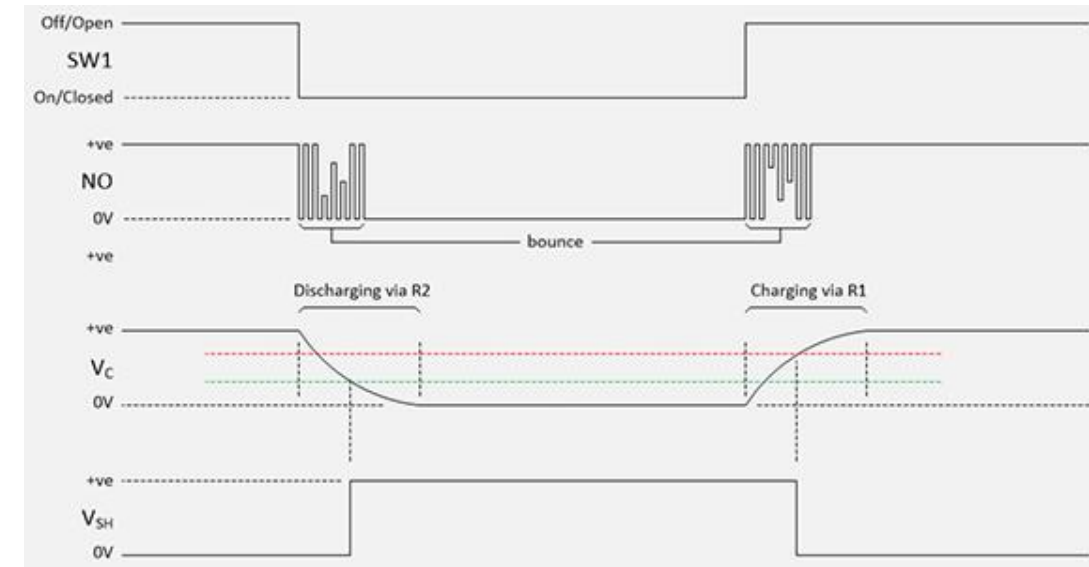
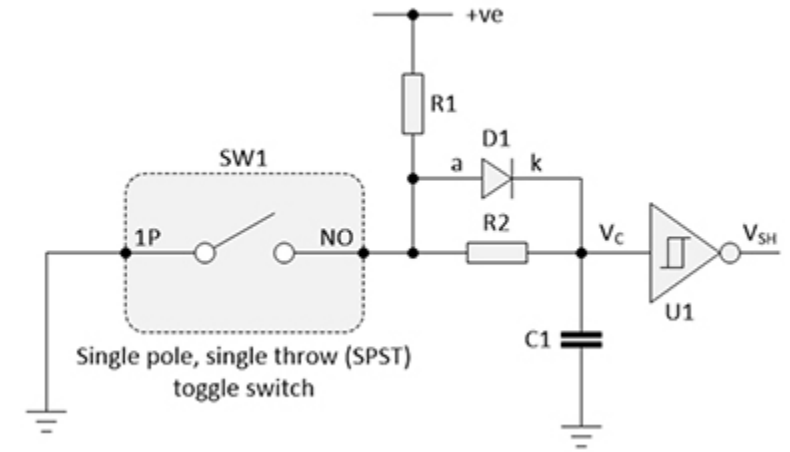
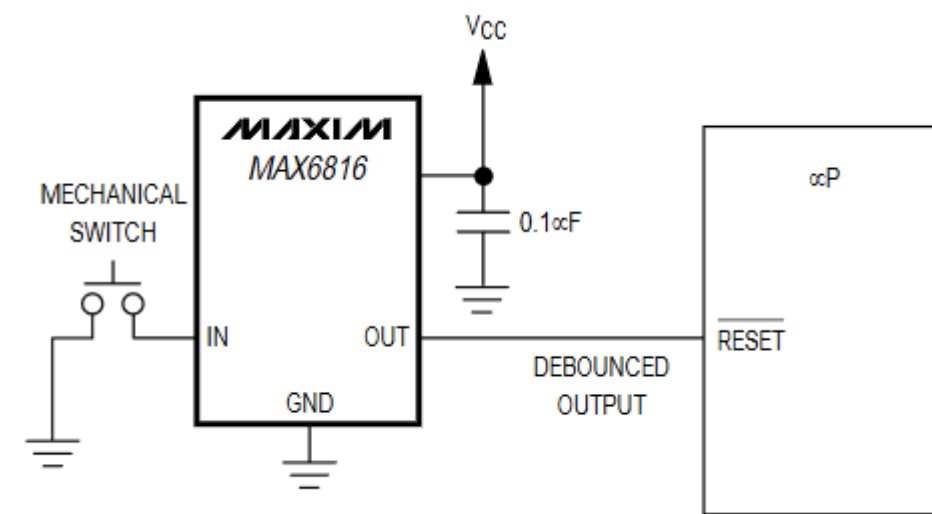
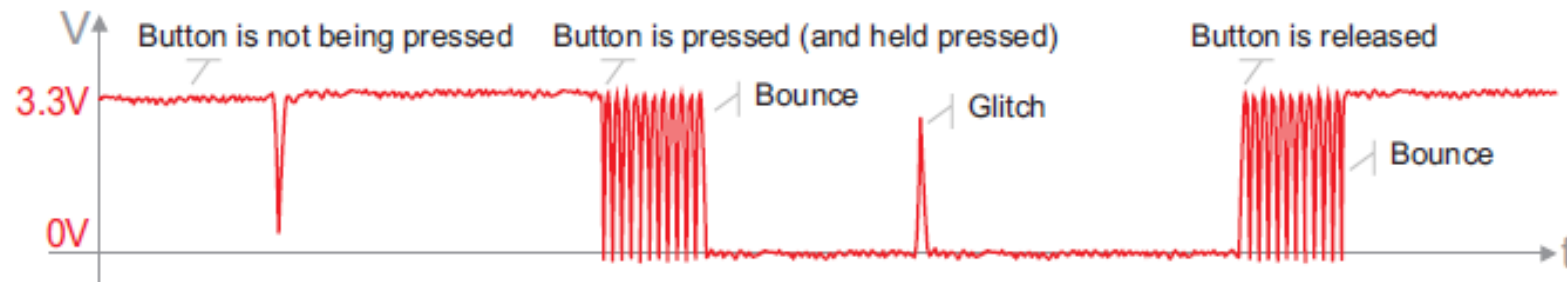


Figura obtenida de: <https://www.digikey.com/es/articles/how-to-implement-hardware-debounce-for-switches-and-relays>

5.2. Repaso: MEF.



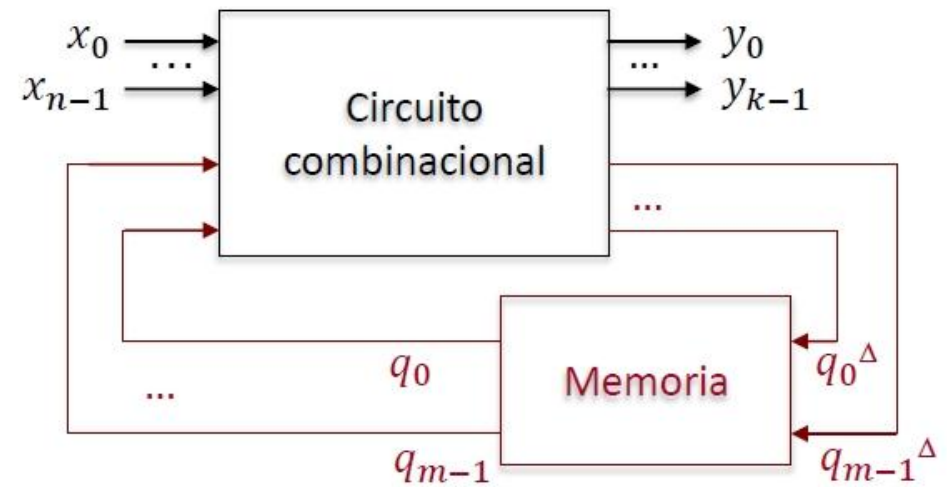
- Normalmente, una fluctuación dura menos de 1 milisegundo, mientras que un rebote puede durar hasta 30 milisegundos.



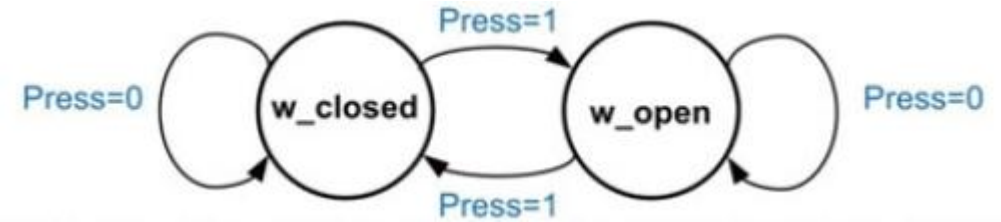
- Existen diferentes hardware para el "debounce", nosotros procesaremos la señal mediante una MEF para distinguir entre la pulsación de un botón y una fluctuación o rebote.

5.2. Repaso: MEF.

- Dos usos de MEF para embebidos:
 - 1- Una MEF es una técnica de organización de software donde el sistema se encuentra en un estado específico y solo cambia ante eventos definidos.
 - 2- Una MEF es una técnica que nos permite desarrollar la estructura de los circuitos lógicos secuenciales más avanzados.
- Aporta orden y robustez a los programas que gestionan múltiples tareas.

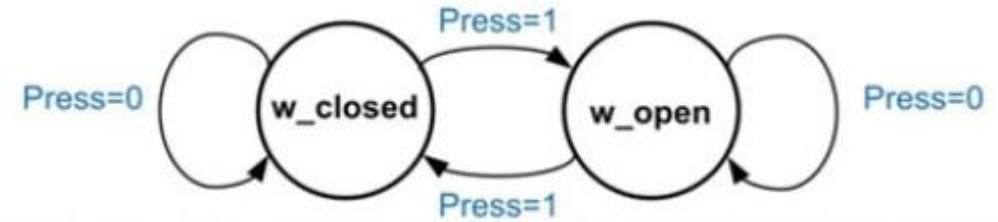


5.2. Repaso: MEF.



- Una MEF contiene un número predefinido de Estados (la máquina puede tener un solo estado).
- La MEF cambia de estados al producirse un evento de activación; por medio de un pulso de reloj o valores de entrada de la máquina.
- El Número de estados y las transiciones están predefinidas mediante una tabla de estados y una secuencia predefinida de transiciones.

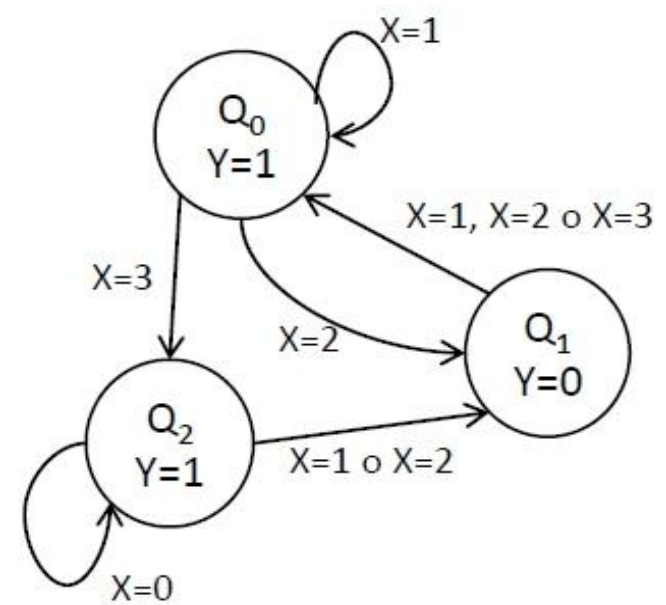
5.2. Repaso: MEF.



- La MEF toma decisiones sobre el estado siguiente (al estado que debe realizar la transición) basándose en un historial de estados pasados.
- Esto permite que la lógica contenga memoria y una salidas más “inteligentes” en comparación con otros circuito lógico combinacional simple, ya que este último tiene salidas basadas solo en los valores actuales de las entradas.

5.2. Repaso: MEF.

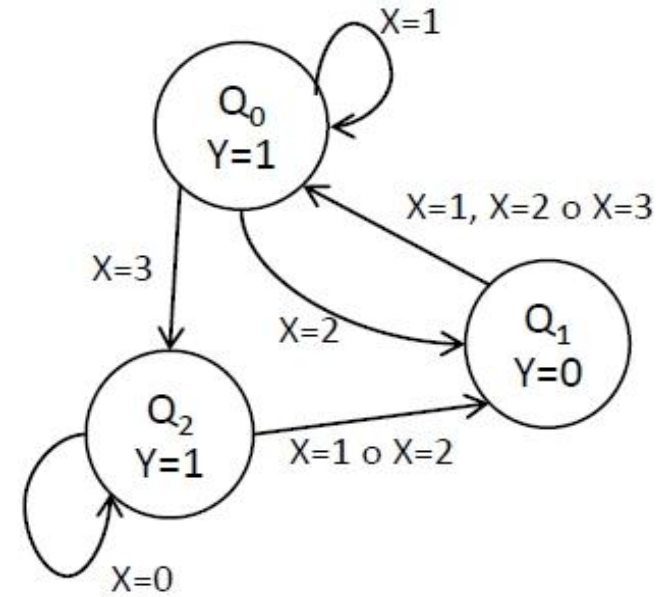
- **Herramientas para el análisis de sistemas discretos se encuentran:** automatización temporizada, álgebra de proceso, redes de Petri y las MEF.



- La MEF surge como un modelo computacional para el diseño de lenguajes de programación (Mealy, 1955; Moore, 1956).
- La MEF describe paso a paso la secuencia o el proceso que se desea implementar, incluyendo las acciones, las condiciones que permiten determinar si el paso ha terminado y si se puede proseguir al siguiente (transiciones entre etapas o estados).

5.2. Repaso: MEF.

- Desventaja; Puede ser el gran número de estados que deban definirse para un proceso muy largo.



- Diversos trabajos han propuesto métodos para simplificar o reducir la máquina de estados.
- Esto no se aborda en esta asignatura.

5.2. Repaso: MEF.

- En general, la estrategia de diseño de un sistema se basa en la modularidad;
- Se propone una MEF para cada subsistema, y se diseña la coordinación de las máquinas individuales mediante otra MEF de nivel superior.
- La MEF con nivel más alto logra la coordinación y supervisión de todos los subsistemas.
- La ventaja de esta estrategia es mantener MEF pequeñas que faciliten la visualización del proceso.

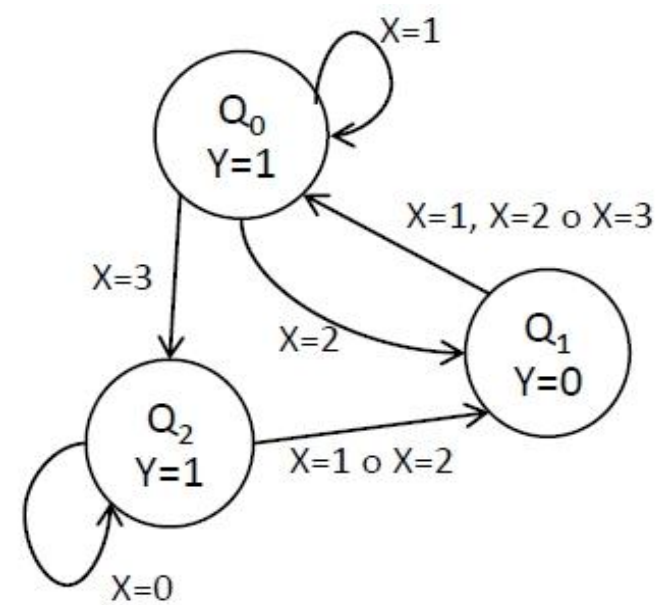


Diagrama de Estados

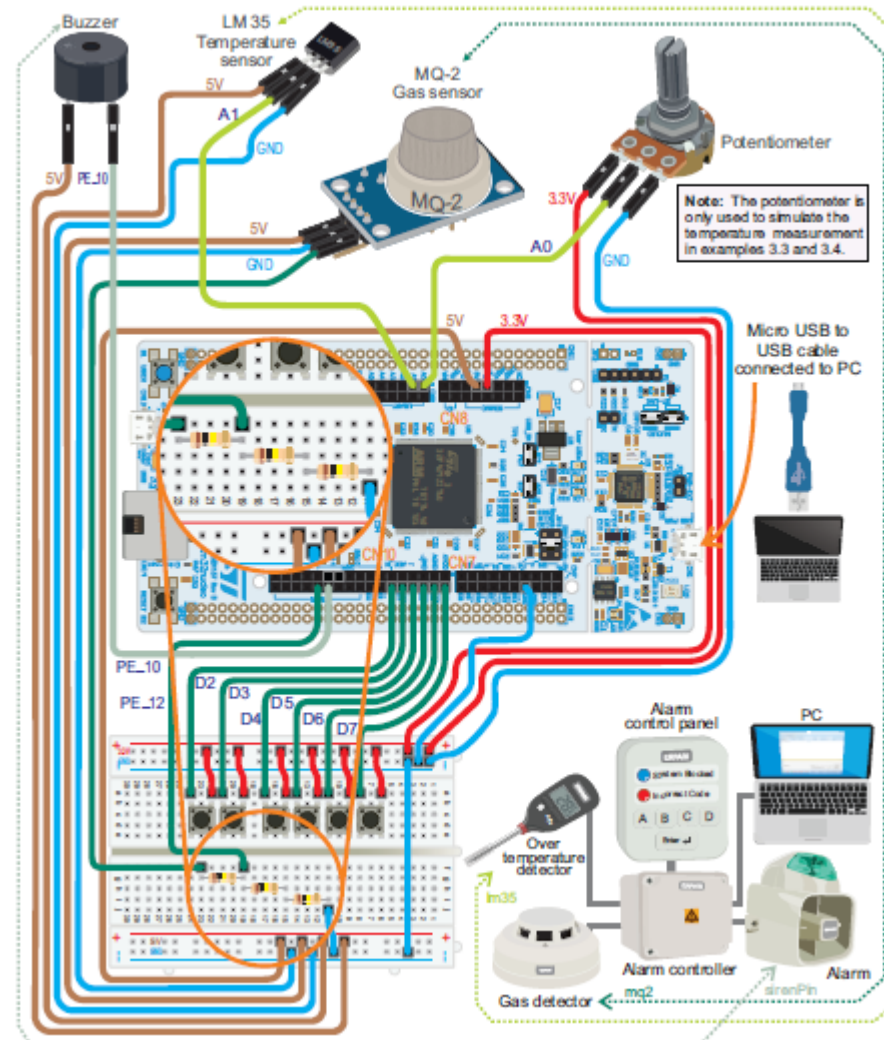
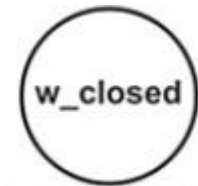


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

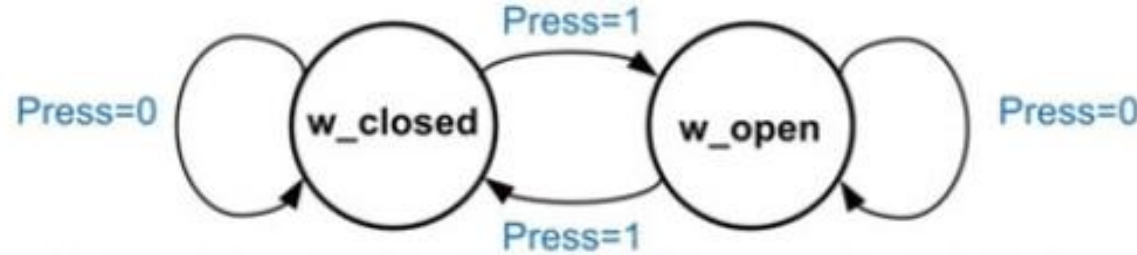
<https://www.arm.com/resources/education/books/designing-embedded-systems>

5.3 Diagrama de estado.

- Es una forma gráfica que describe el funcionamiento de un MEF.
- Es un gráfico dirigido, en el que cada estado (o vértice) dentro del sistema se denota con un círculo y se le da un nombre descriptivo.
- Los nombres están escritos dentro de los círculos.
- Las transiciones entre estados se indican mediante flechas con las condiciones de entrada que causan las transiciones escritas a su lado.



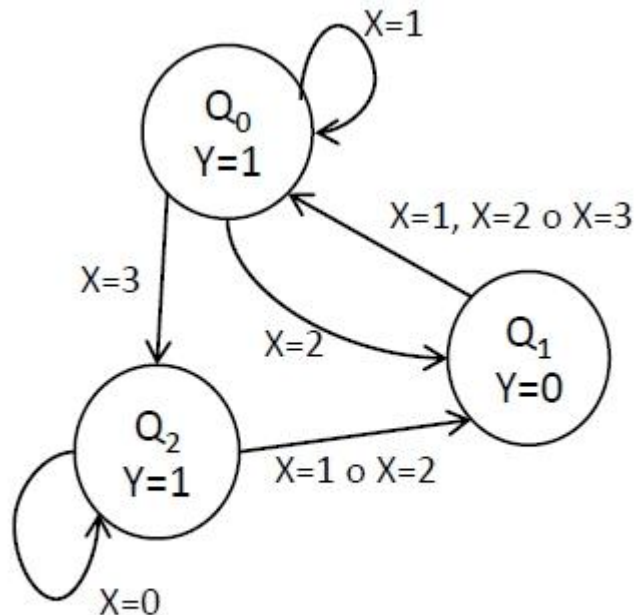
5.3 Diagrama de estado.



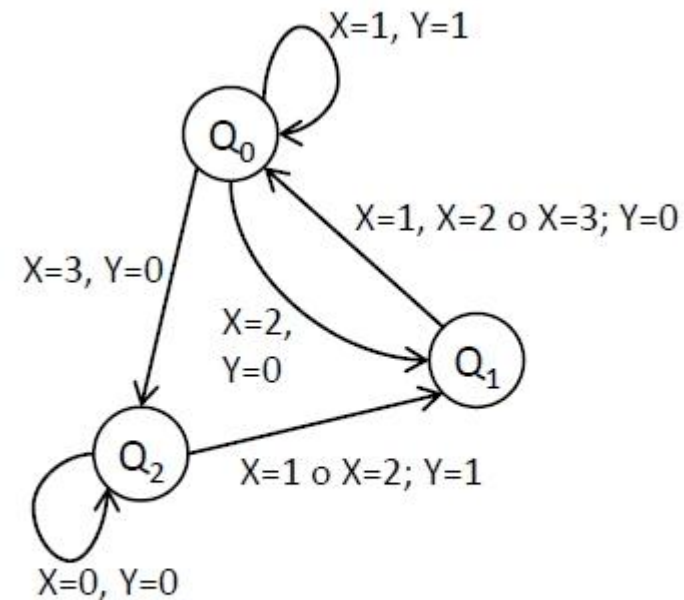
- Las transiciones permite moverse a diferentes estados en condiciones de entrada particulares o permanecer en el mismo estado.
- Una MEF activa una evaluación para realizar una transición de estados cada vez que los dispositivos de almacenamiento actualizan sus salidas.

5.3 Diagrama de estado.

- Mealy: las salidas están en función de dos, el estado presente y las entrada.
- Moore: Las salidas están en función del estado presente solamente.



MÁQUINA DE MOORE



MÁQUINA DE MEALY

Problemas

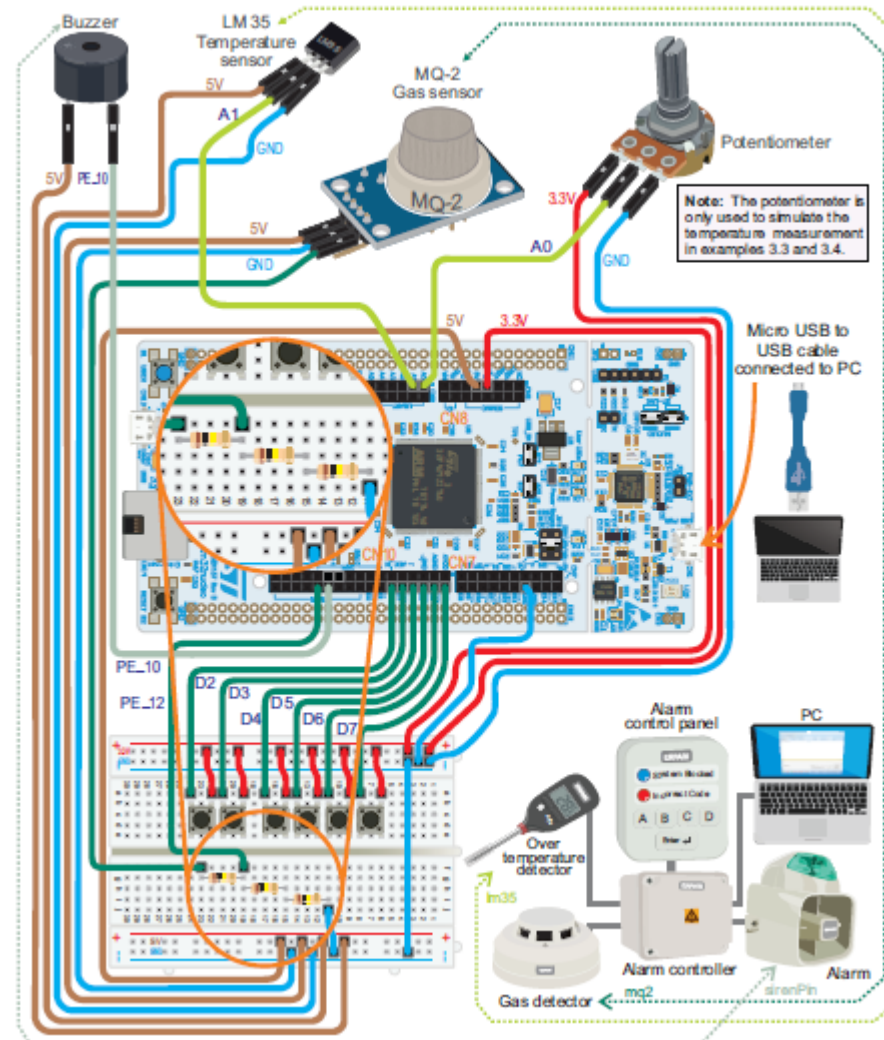


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

<https://www.arm.com/resources/education/books/designing-embedded-systems>

5.4 Problemas.

- Ejemplo 4.1: MEF para un Botón de Usuario.

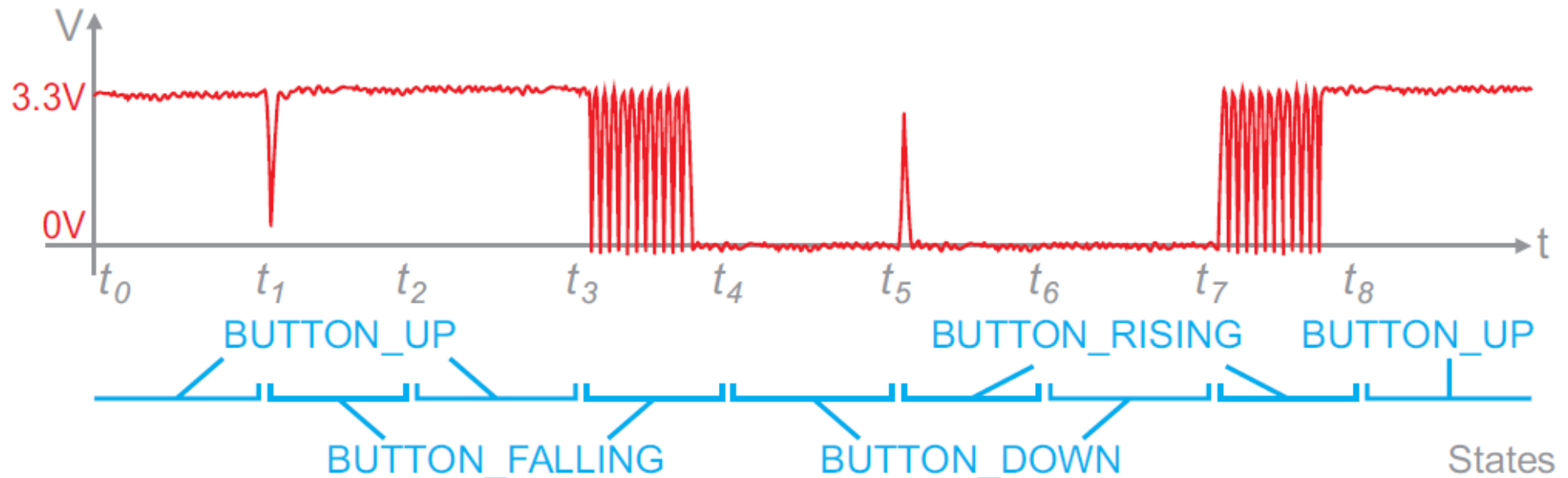


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, <https://www.arm.com/resources/education/books/designing-embedded-systems>

5.4 Problemas.

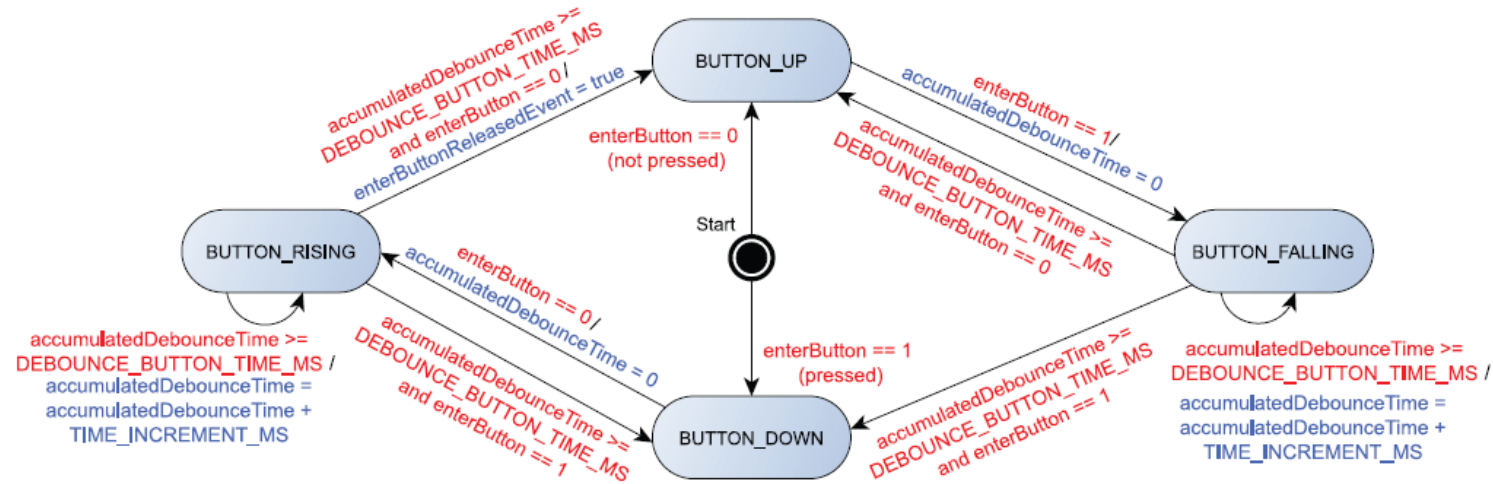
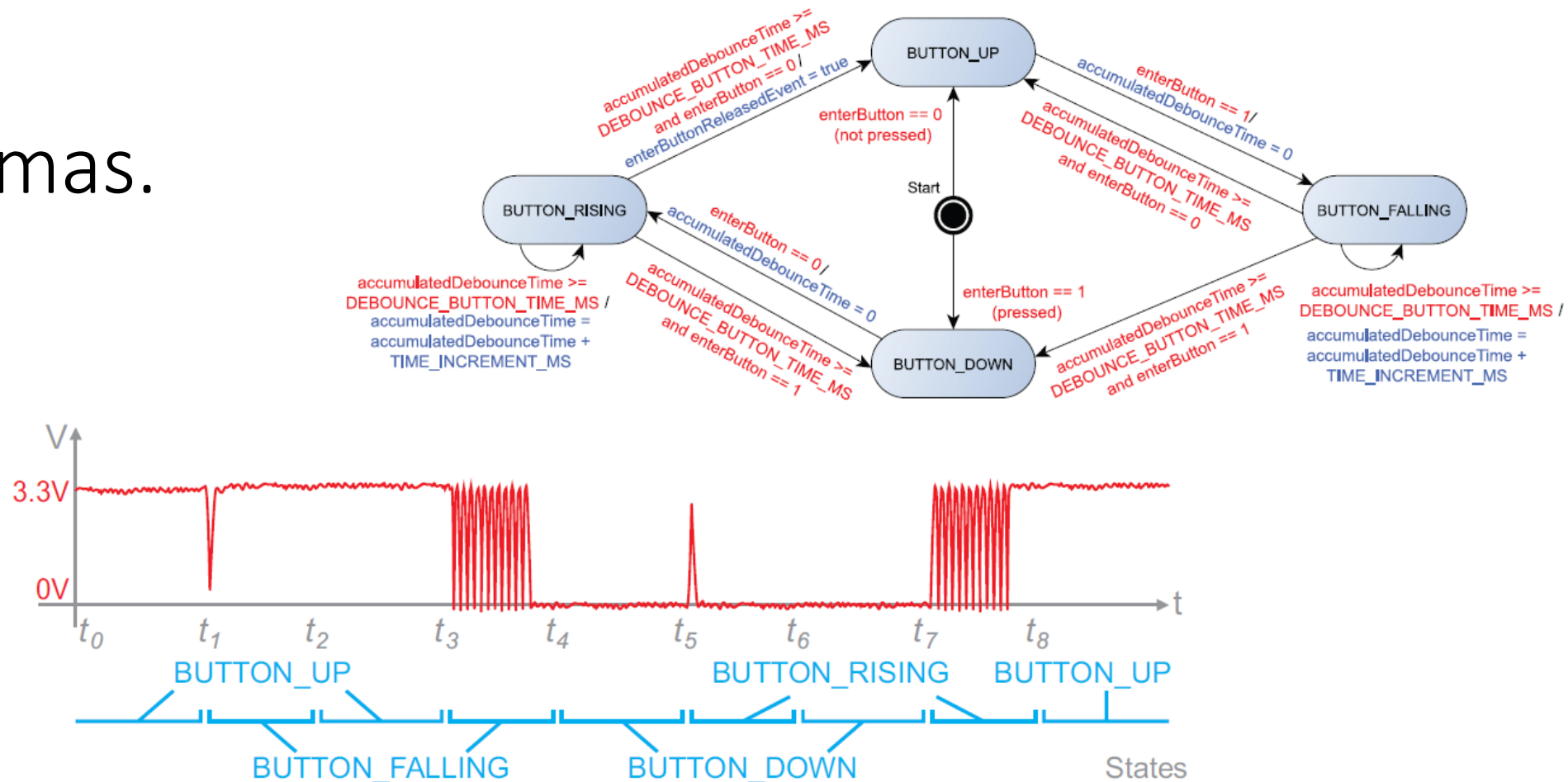


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, <https://www.arm.com/resources/education/books/designing-embedded-systems>

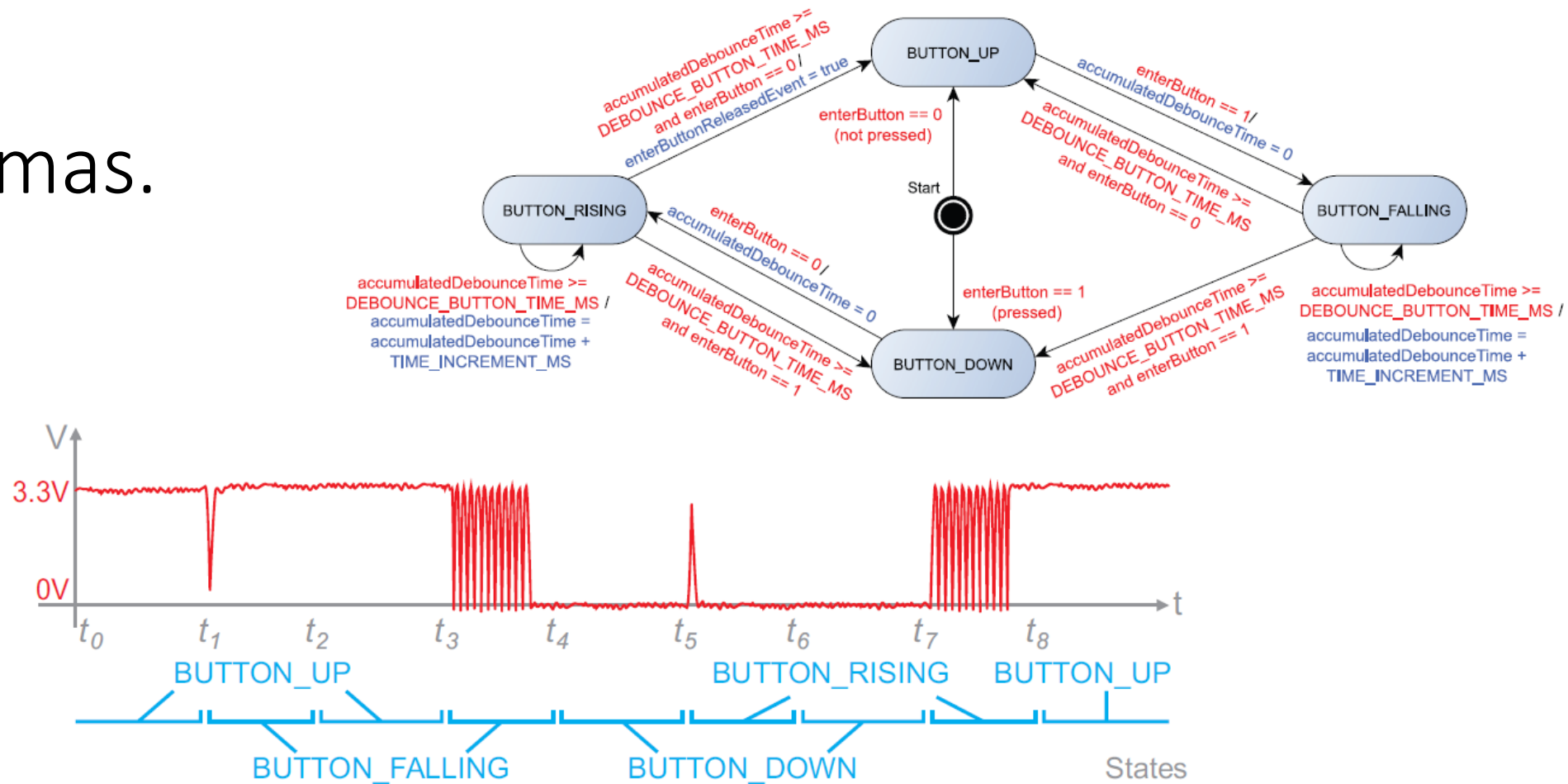
- Representación gráfica de una máquina de estados finitos.
- Es común desarrollar el diagrama de la MEF para analizar y ajustar el comportamiento del sistema, para finalmente implementarla.

5.4 Problemas.



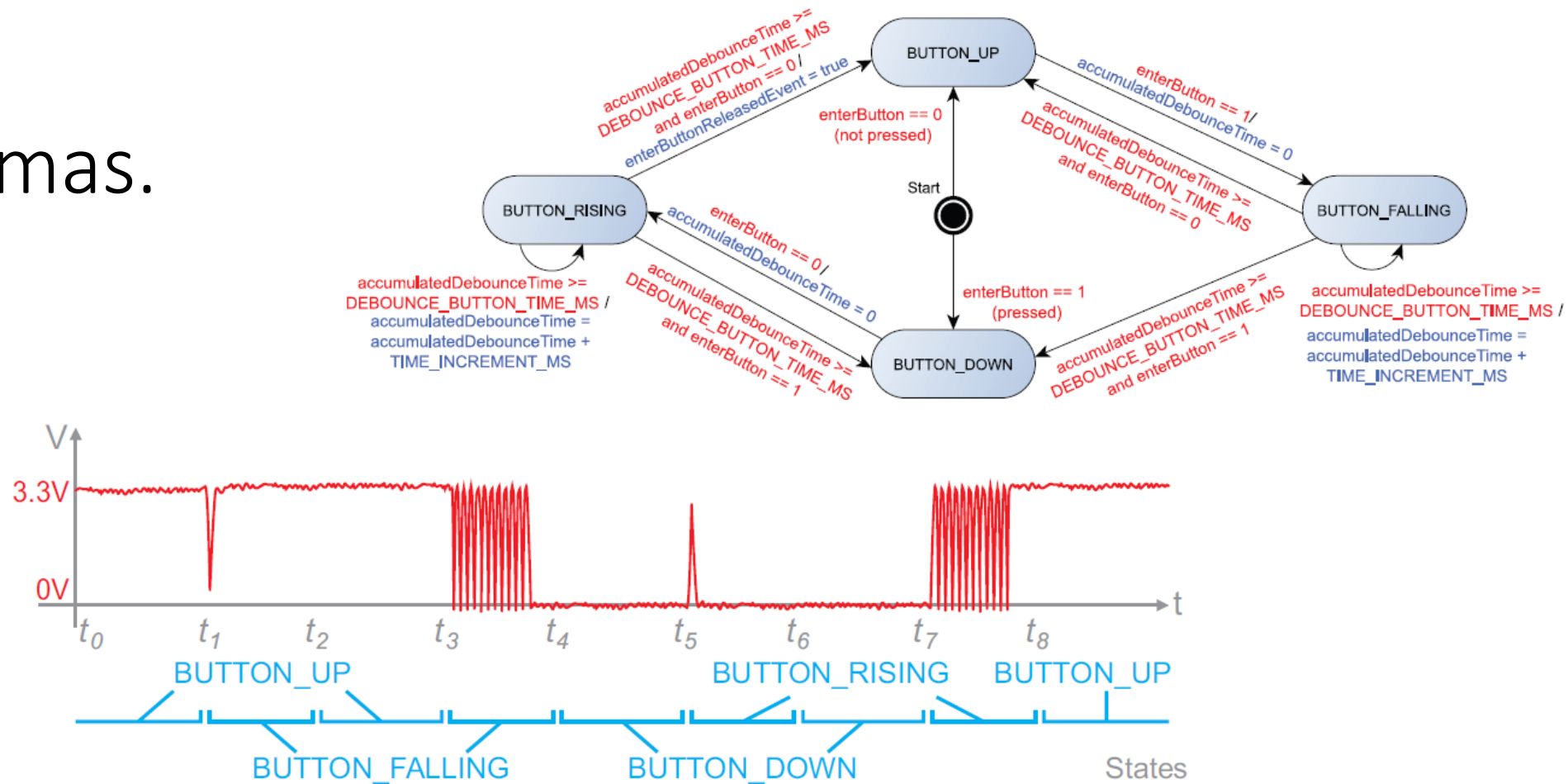
- En t_0 el botón está suelto (3,3 V) y la MEF se encuentra en **BUTTON_UP**.
- En t_1 se produce una anomalía, la MEF cambia a **BUTTON_FALLING** y luego regresa a **BUTTON_UP** en t_2 .

5.4 Problemas.



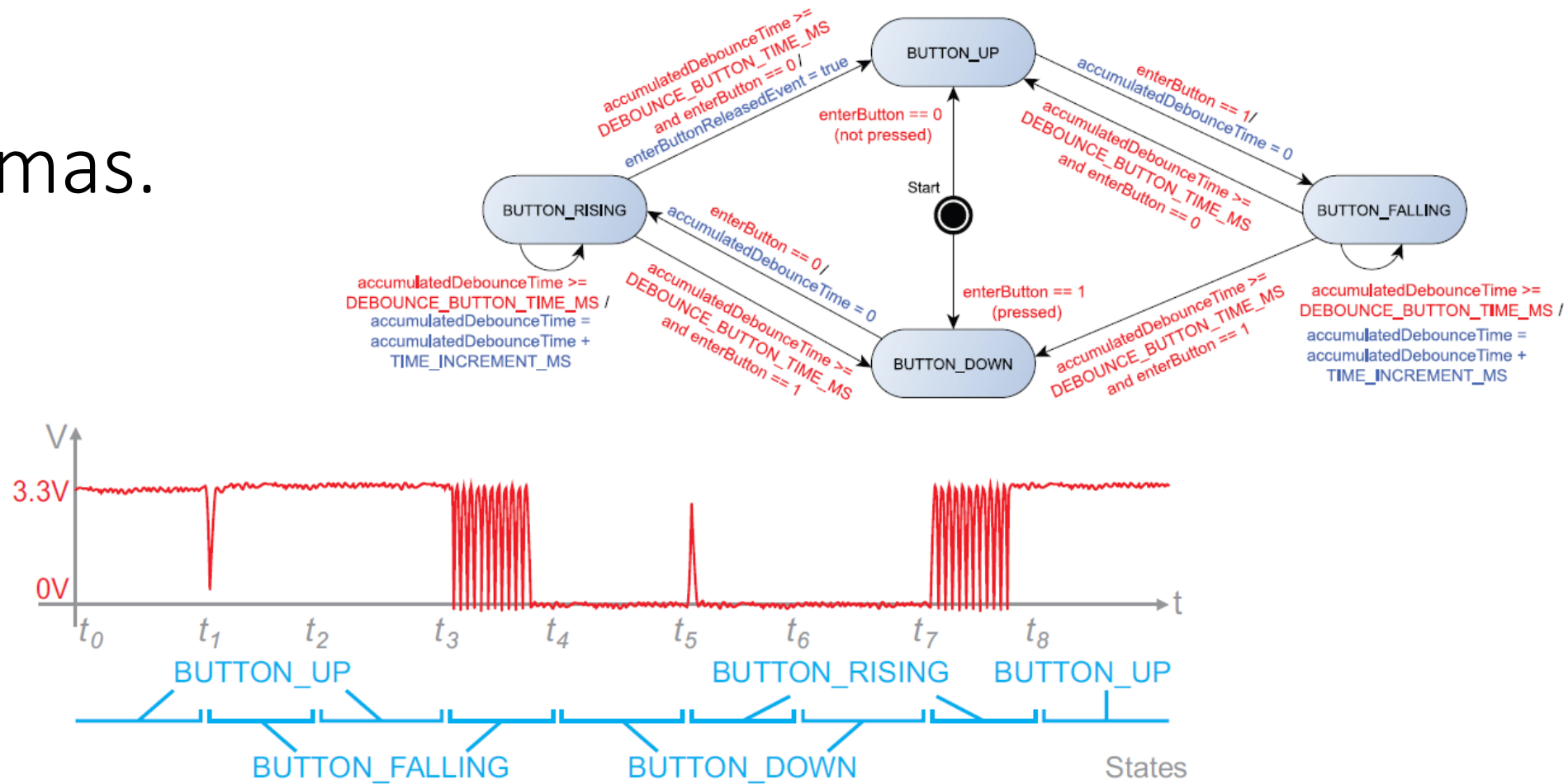
- En t_3 se observa un rebote debido a que se presiona el botón, la MEF cambia a **BUTTON_FALLING**.
- En t_4 , el estado de la FSM cambia a **BUTTON_DOWN**, ya que la señal de voltaje se mantiene estable en 0 V.

5.4 Problemas.



- En t_5 se produce una nueva anomalía, tras la cual la MEF cambia a **BUTTON_RISING**.
- Como la señal de voltaje permanece en 0 V, el estado de la FSM regresa a **BUTTON_DOWN** en t_6 .

5.4 Problemas.



- En t_7 se produce una transición de botón liberado a pulsado, acompañada de un rebote en la señal.
- La MEF cambia a **BUTTON_RISING**, la señal se estabiliza y en t_8 el estado de la FSM cambia a **BUTTON_UP**.

5.4 Problemas.

- En el problema se definen cuatro pines, tres como entrada y uno como salida. Las entradas representan la salida digital un sensor de gas y otro de temperatura.

La salida representa la activación de una alarma.

El botón reset apaga la alarma si se acciono.

- En la figura se explica la lógica del problema.

Extractos del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers (pag.19)
<https://www.arm.com/resources/education/books/designing-embedded-systems>

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
```

Include the libraries "mbed.h" and "arm_book_lib.h"

```
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn alarmOffButton(BUTTON1);
9
10    DigitalOut alarmLed(LED1);
11
12    gasDetector.mode(PullDown);
13    overTempDetector.mode(PullDown);
14
15    alarmLed = OFF;
16
17    bool alarmState = OFF;
18
19    while ( true ) {
20
21        if ( gasDetector || overTempDetector ) {
22            alarmState = ON;
23        }
24
25        alarmLed = alarmState;
26
27        if ( alarmOffButton ) {
28            alarmState = OFF;
29        }
30    }
31 }
```

- Digital input objects named *gasDetector*, *overTempDetector* and *alarmOffButton*, are declared and assigned to D2, D3, and BUTTON1 (B1 USER), respectively.

- Digital output object named *alarmLed* is declared and assigned to LED1.

- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.

- *alarmLed* is assigned OFF.

- *alarmState* is declared and assigned OFF.

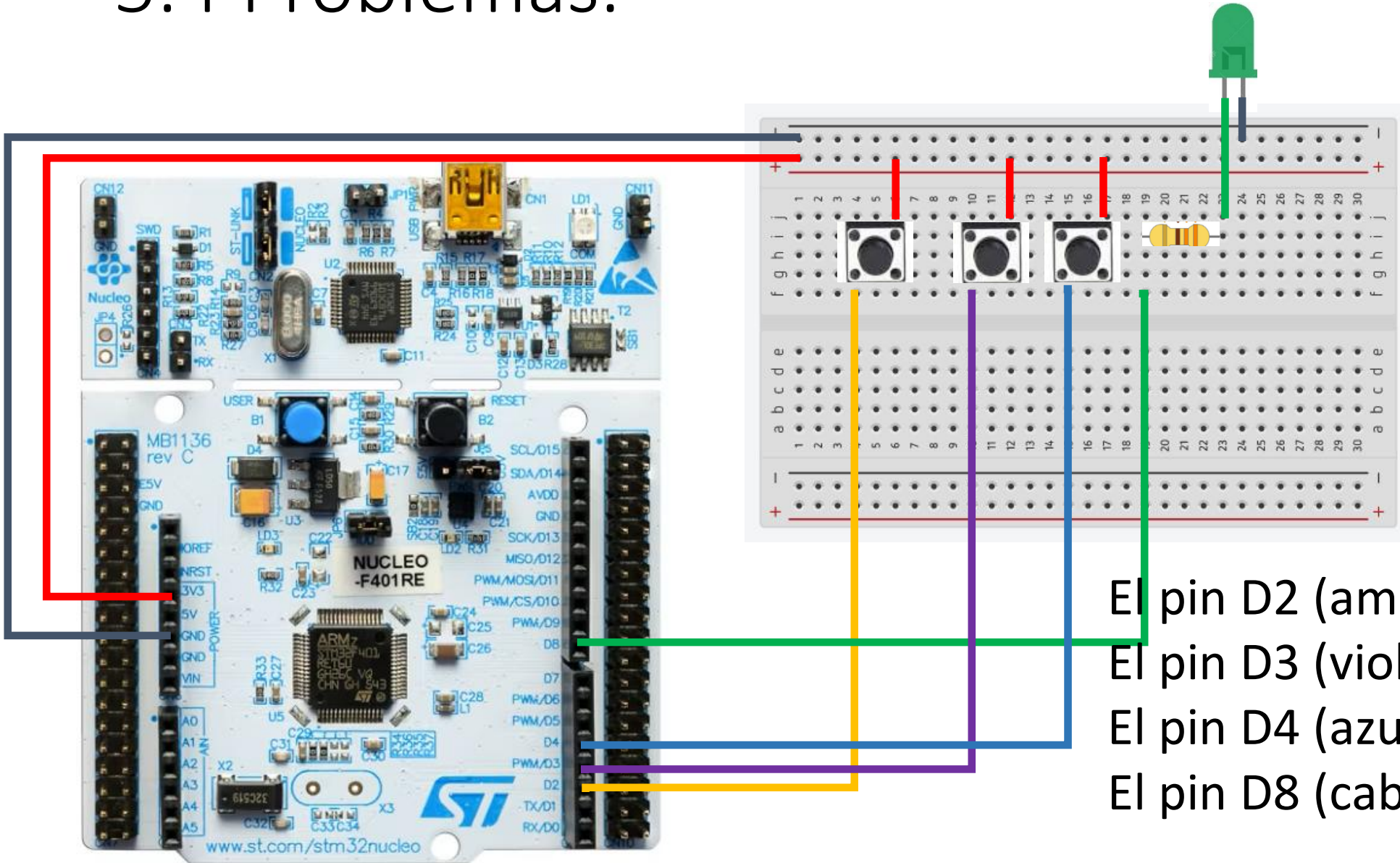
Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.

- Assign *alarmLed* with *alarmState*.

- If *alarmOffButton*, assign *alarmState* with OFF.

5.4 Problemas.



El pin D2 (amarillo) es PA10.
El pin D3 (violeta) es PB3.
El pin D4 (azul) es PB5.
El pin D8 (cable verde) es PA9.

5.4 Problemas.

```
#include <stdbool.h>
#include "main.h"
#include "gpio.h"
void SystemClock_Config(void);
int main(void) {
    bool alarmState=0;
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    HAL_GPIO_WritePin(led_GPIO_Port, led_Pin, alarmState);
    HAL_Delay(100);
    while (1){
        if(HAL_GPIO_ReadPin(sensor_gas_GPIO_Port,sensor_gas_Pin)||HAL_GPIO_ReadPin(sensor_
temp_GPIO_Port,sensor_temp_Pin)){
            alarmState=1;
        }
        HAL_GPIO_WritePin(led_GPIO_Port, led_Pin, alarmState);
        if(HAL_GPIO_ReadPin(boton_GPIO_Port, boton_Pin)){
            alarmState=0;
        }
    }
}
```

```
1 #include "mbed.h"
2 #include "arm_book_lib.h"
3
4 int main()
5 {
6     DigitalIn gasDetector(D2);
7     DigitalIn overTempDetector(D3);
8     DigitalIn alarmOffButton(BUTTON1);
9
10    DigitalOut alarmLed(LED1);
11
12    gasDetector.mode(PullDown);
13    overTempDetector.mode(PullDown);
14
15    alarmLed = OFF;
16
17    bool alarmState = OFF;
18
19    while ( true ) {
20
21        if ( gasDetector || overTempDetector ) {
22            alarmState = ON;
23        }
24
25        alarmLed = alarmState;
26
27        if ( alarmOffButton ) {
28            alarmState = OFF;
29        }
30    }
31 }
```

Include the libraries "mbed.h"
and "arm_book_lib.h"

- Digital input objects named *gasDetector*, *overTempDetector* and *alarmOffButton*, are declared and assigned to D2, D3, and BUTTON1 (B1 USER), respectively.
- Digital output object named *alarmLed* is declared and assigned to LED1.
- *gasDetector* and *overTempDetector* are configured with internal pull-down resistors.
- *alarmLed* is assigned OFF.
- *alarmState* is declared and assigned OFF.

Do forever loop:

- If *gasDetector* or *overTempDetector* are active, assign *alarmState* with ON.
- Assign *alarmLed* with *alarmState*.
- If *alarmOffButton*, assign *alarmState* with OFF.

5.4 Problemas.

- MEF para apagar la alarma.
- Devuelve un bool.
- Al detectar el pulso del usuario la variable **enterButtonReleasedEvent** entrega un true.

```
1  bool debounceButtonUpdate()  
2  {  
3      bool enterButtonReleasedEvent = false;  
4      switch( enterButtonState ) {  
5  
6          case BUTTON_UP:  
7              if( enterButton ) {  
8                  enterButtonState = BUTTON_FALLING;  
9                  accumulatedDebounceButtonTime = 0;  
10             }  
11             break;  
12  
13             case BUTTON_FALLING:  
14                 if( accumulatedDebounceButtonTime >= DEBOUNCE_BUTTON_TIME_MS ) {  
15                     if( enterButton ) {  
16                         enterButtonState = BUTTON_DOWN;  
17                     } else {  
18                         enterButtonState = BUTTON_UP;  
19                     }  
20                 }  
21                 accumulatedDebounceButtonTime = accumulatedDebounceButtonTime +  
22                     TIME_INCREMENT_MS;  
23                 break;  
24  
25             case BUTTON_DOWN:  
26                 if( !enterButton ) {  
27                     enterButtonState = BUTTON_RISING;  
28                     accumulatedDebounceButtonTime = 0;  
29                 }  
30                 break;  
31  
32             case BUTTON_RISING:  
33                 if( accumulatedDebounceButtonTime >= DEBOUNCE_BUTTON_TIME_MS ) {  
34                     if( !enterButton ) {  
35                         enterButtonState = BUTTON_UP;  
36                         enterButtonReleasedEvent = true;  
37                     } else {  
38                         enterButtonState = BUTTON_DOWN;  
39                     }  
40                 }  
41                 accumulatedDebounceButtonTime = accumulatedDebounceButtonTime +  
42                     TIME_INCREMENT_MS;  
43                 break;  
44  
45             default:  
46                 debounceButtonInit();  
47                 break;  
48             }  
49             return enterButtonReleasedEvent;  
50     }
```


5.4 Problemas.

```
bool debounceButtonUpdate(void) {
    bool eventReleased = false;
    switch (buttonState) {

    case BUTTON_UP:
        if (isButtonPressed()){
            buttonState = BUTTON_FALLING;
            accumulatedDebounceTime = 0;
        } break;

    case BUTTON_FALLING:
        if (accumulatedDebounceTime >= DEBOUNCE_TIME){
            if (isButtonPressed()){
                buttonState = BUTTON_DOWN;
            } else {
                buttonState = BUTTON_UP;
            }
            accumulatedDebounceTime += TIME_INCREMENT;
            break;
        }
    }
```

```
case BUTTON_DOWN:
    if (!isButtonPressed()){
        buttonState = BUTTON_RISING;
        accumulatedDebounceTime = 0; }
    break;

case BUTTON_RISING:
    if (accumulatedDebounceTime >= DEBOUNCE_TIME){
        if (!isButtonPressed()) {
            buttonState = BUTTON_UP;
            eventReleased = true;
        } else {
            buttonState = BUTTON_DOWN;
        }
        accumulatedDebounceTime += TIME_INCREMENT;
        break; }
    return eventReleased;
}
```

5.4 Problemas.

```
#include "main.h"
#include "gpio.h"
#include <stdbool.h>
#define DEBOUNCE_TIME 50
#define TIME_INCREMENT 10
static int pulsacionesValidas = 0;
typedef enum {
    BUTTON_UP,
    BUTTON_FALLING,
    BUTTON_DOWN,
    BUTTON_RISING
} buttonState_t;

buttonState_t buttonState = BUTTON_UP;

int accumulatedDebounceTime = 0;
bool isButtonPressed(void);
bool debounceButtonUpdate(void);
void SystemClock_Config(void);
```

```
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_TIM2_Init();
    while (1) {
        if (debounceButtonUpdate()) {
            pulsacionesValidas++;
        }
        HAL_Delay(TIME_INCREMENT);
    }

    bool isButtonPressed(void) {

        return (HAL_GPIO_ReadPin(boton_GPIO_Port,
            boton_Pin) == GPIO_PIN_RESET); }
}
```

Ciclo de Histéresis con MEF

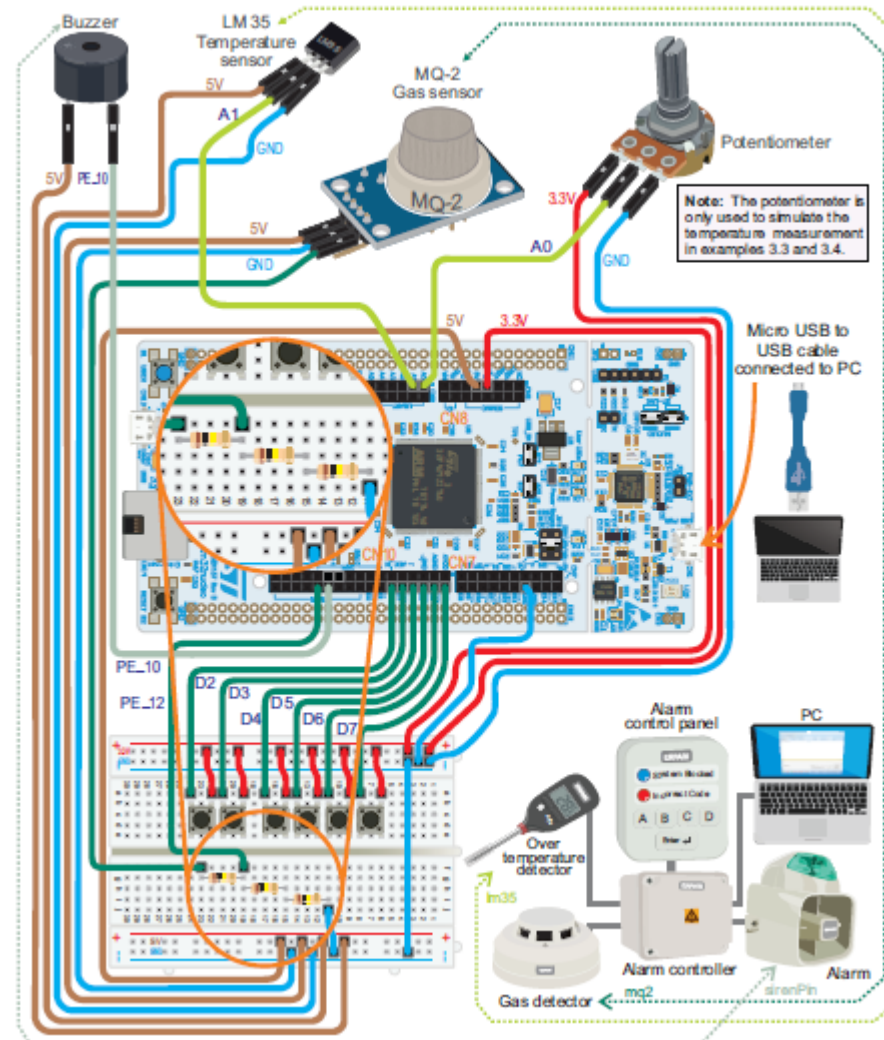
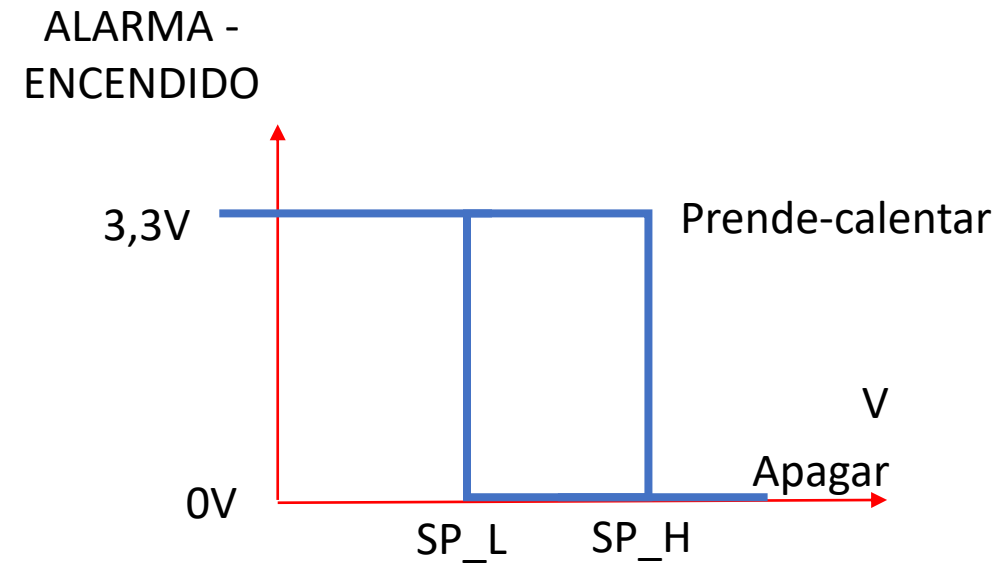


Figura del libro: A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Página 87.

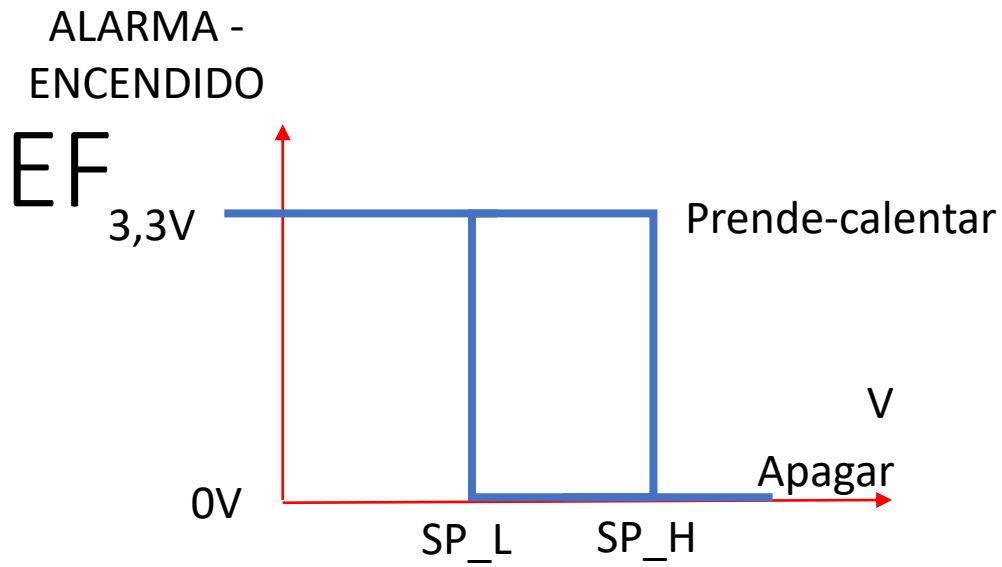
<https://www.arm.com/resources/education/books/designing-embedded-systems>

5.5 Ciclo de Histéresis con MEF



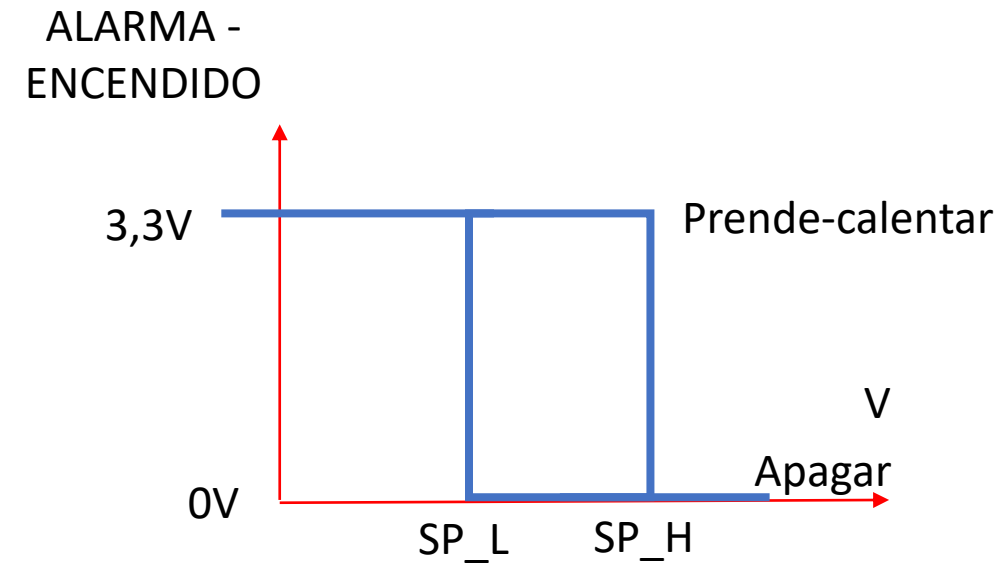
- Técnica de control de lazo cerrado (feedback) que utiliza dos umbrales de referencia en lugar de uno solo para decidir cuándo cambiar el estado de una salida.
- **Función principal:** evitar el "chattering" o conmutación errática cuando una señal de entrada tiene ruido o pequeñas fluctuaciones cerca del punto de consigna (setpoint).

5.5 Ciclo de Histéresis con MEF



- ¿Cómo funciona en un sistema embebido?
- En un sistema embebido, el código actúa como el comparador.
- Medir temperatura para mantener la temperatura de un horno entre dos magnitudes, TL y TH.

5.5 Ciclo de Histéresis con MEF



- Sin Histéresis, el encendido y apagado del horno podría funcionar de forma errática, al querer activar o desactivar la fuente de calor.
- Con Histéresis, el encendido y apagado realiza una acción definida, ya que los puntos de comparación están separados por el ancho de la ventana de histéresis.

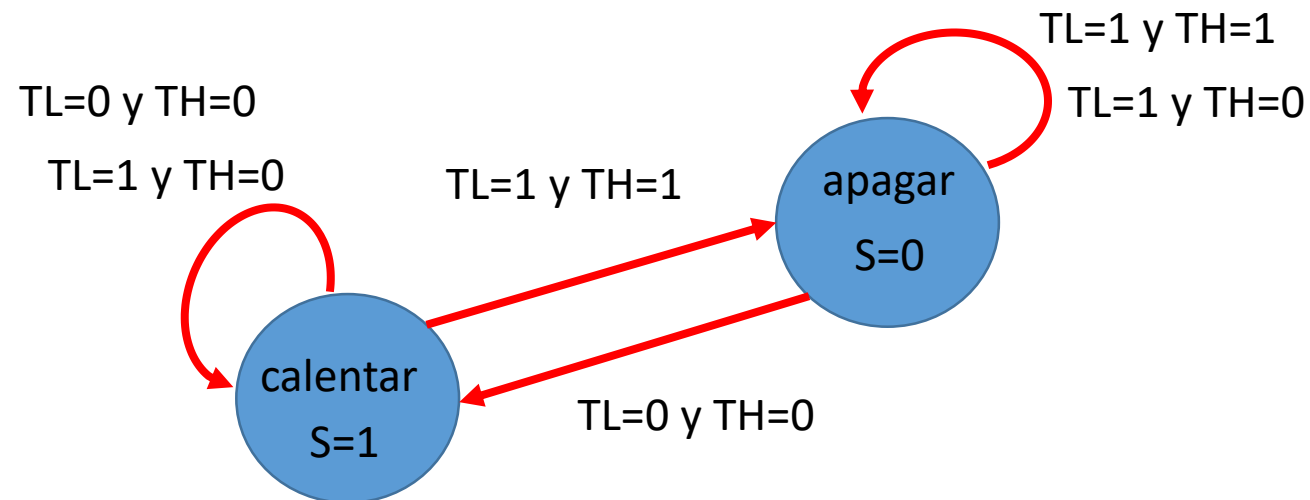
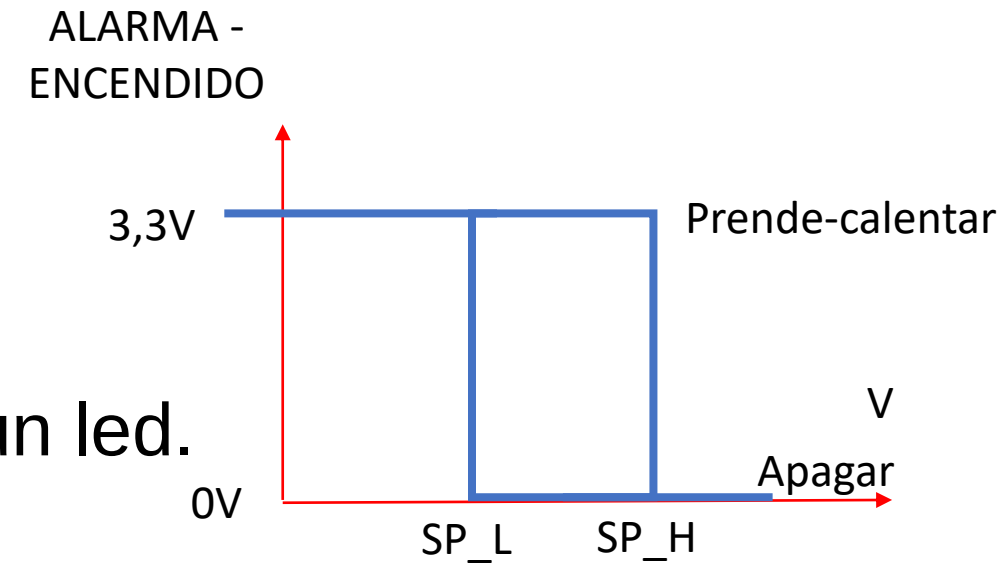
5.5 Ciclo de Histéresis con MEF

- Algunos dispositivos que utilizan esta lógica.

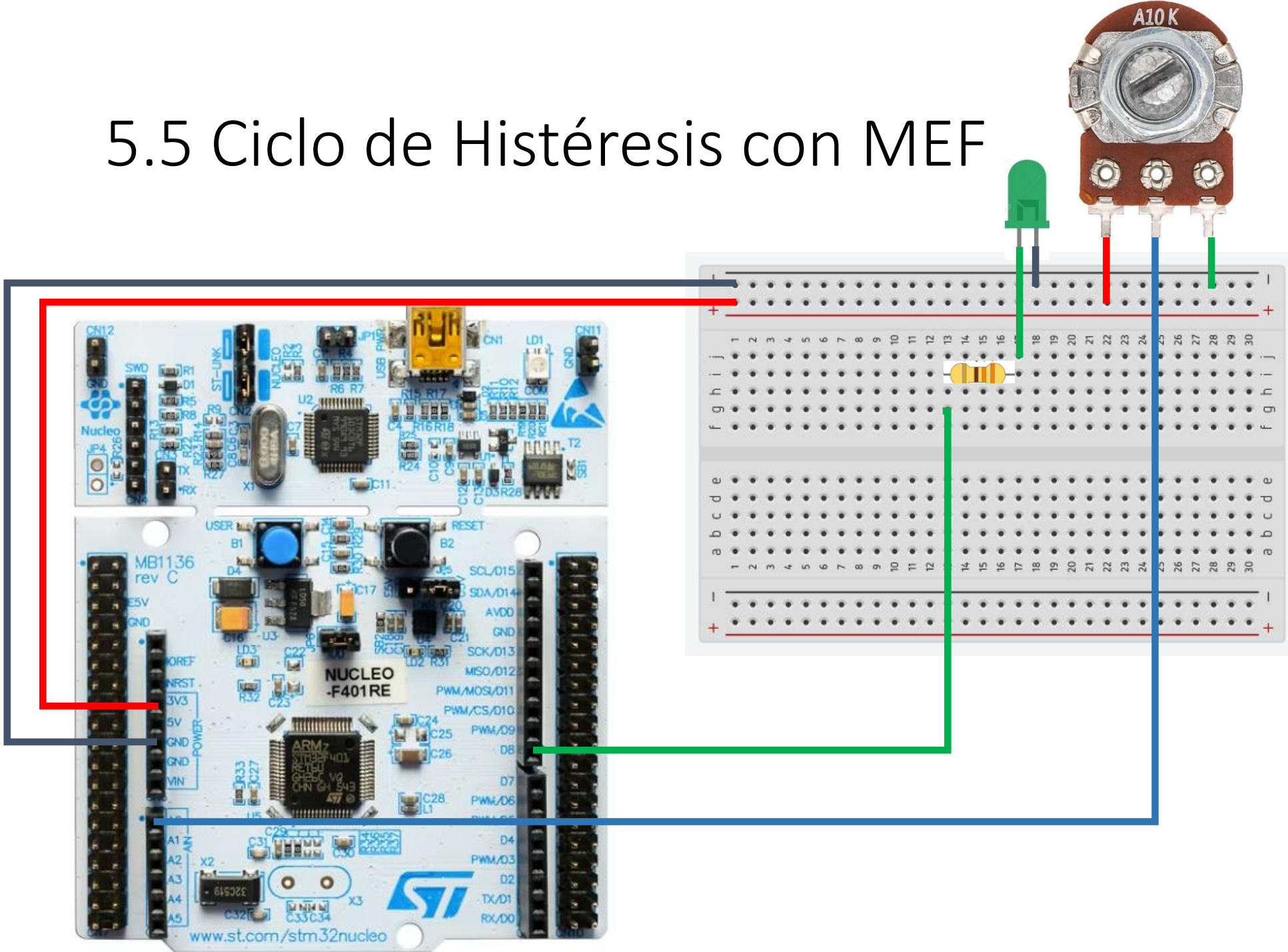


5.5 Ciclo de Histéresis con MEF

- Utilizaremos para prender y apagar un led.
- Representa una alarma o el encendido de una fuente de calor.



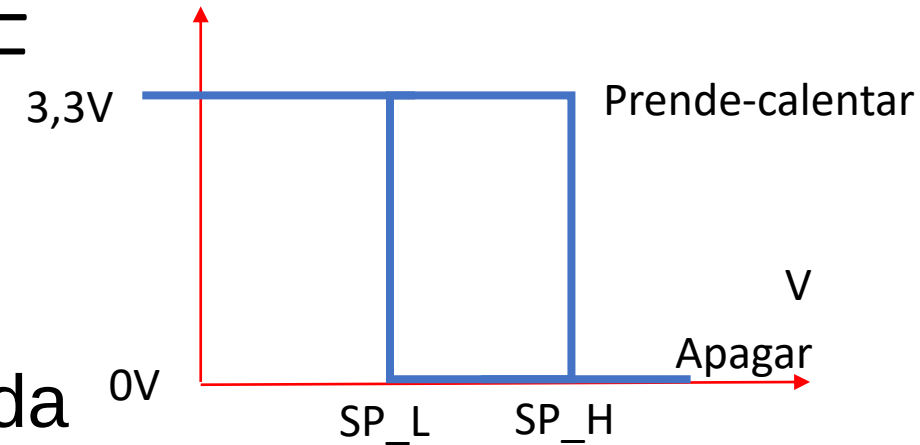
5.5 Ciclo de Histéresis con MEF



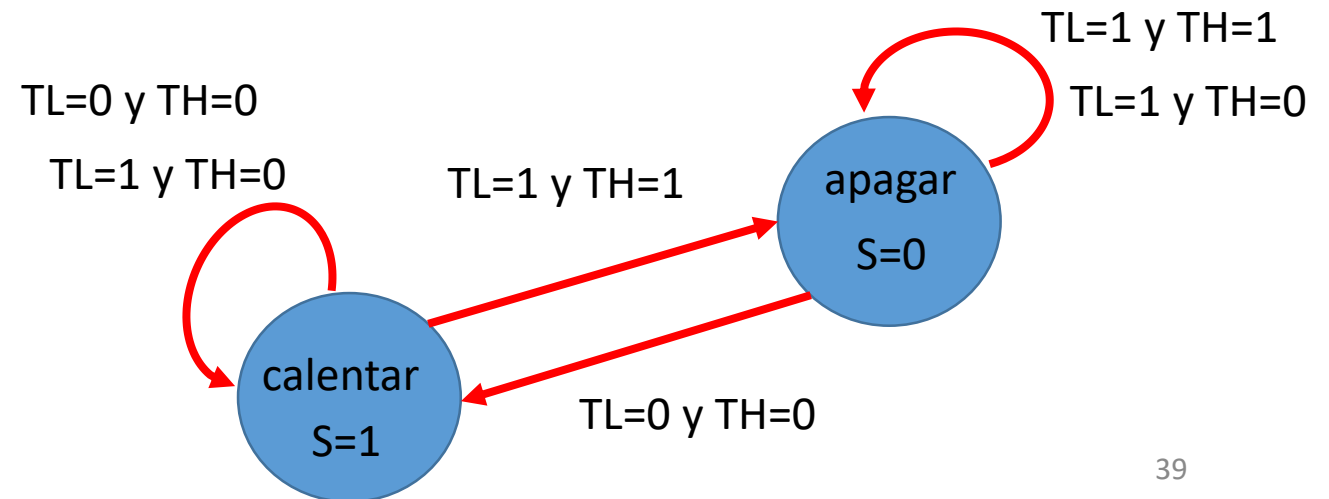
5.5 Ciclo de Histéresis con MEF

- Utilizaremos un potenciómetro como sensor y se compara la magnitud medida con SP_L y SP_H para obtener TH y TL.

ALARMA -
ENCENDIDO



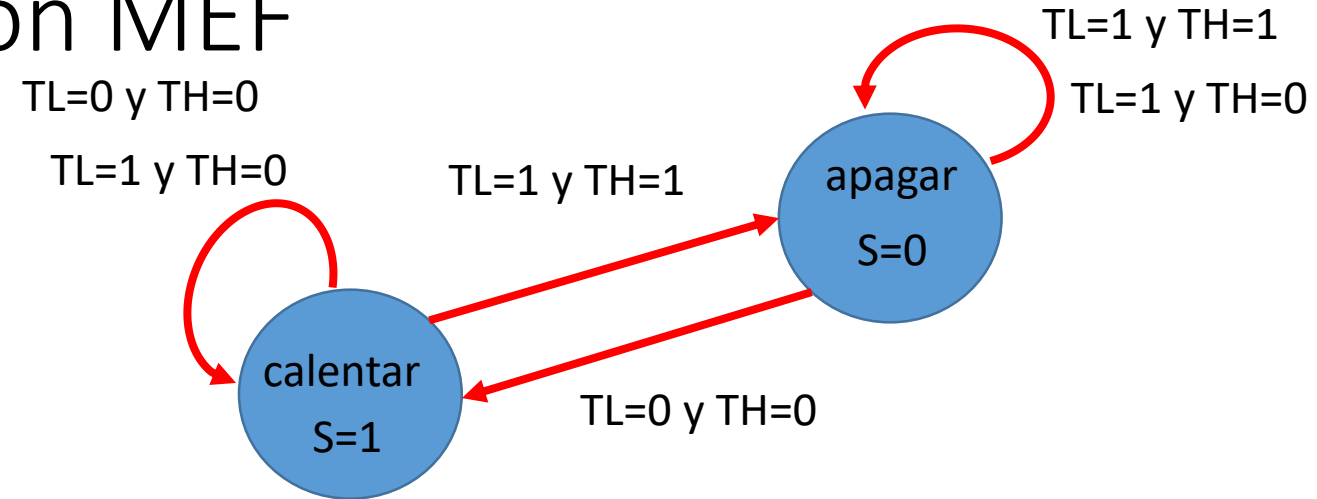
```
void medir(void) {  
    HAL_ADC_Start(&hadc1);  
    if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {  
        adc_raw = HAL_ADC_GetValue(&hadc1);  
        Temp = adc_raw/10.0;  
        if (Temp > SP_H) {  
            TH=1;  
        }else{  
            TH=0;  
        }  
        if (Temp > SP_L) {  
            TL=1;  
        }else{  
            TL=0;  
        }  
    }  
}
```



5.5 Ciclo de Histéresis con MEF

```
typedef enum {  
    CALENTAR,  
    APAGAR  
} estados_sistema;
```

```
void histeresis(void) {  
    medir();  
    switch (estado) {  
        case CALENTAR:  
            HAL_GPIO_WritePin(led_GPIO_Port, led_Pin, GPIO_PIN_SET);  
            if ((TL == 0 && TH==0) || (TL == 1 && TH==0)) {  
                estado = CALENTAR;  
            } else {  
                estado = APAGAR;  
            } break;  
        case APAGAR:  
            HAL_GPIO_WritePin(led_GPIO_Port, led_Pin, GPIO_PIN_RESET);  
            if ((TL == 1 && TH==0) || (TL == 1 && TH==1)) {  
                estado = APAGAR;  
            } else {  
                estado = CALENTAR;  
            } break; } }
```

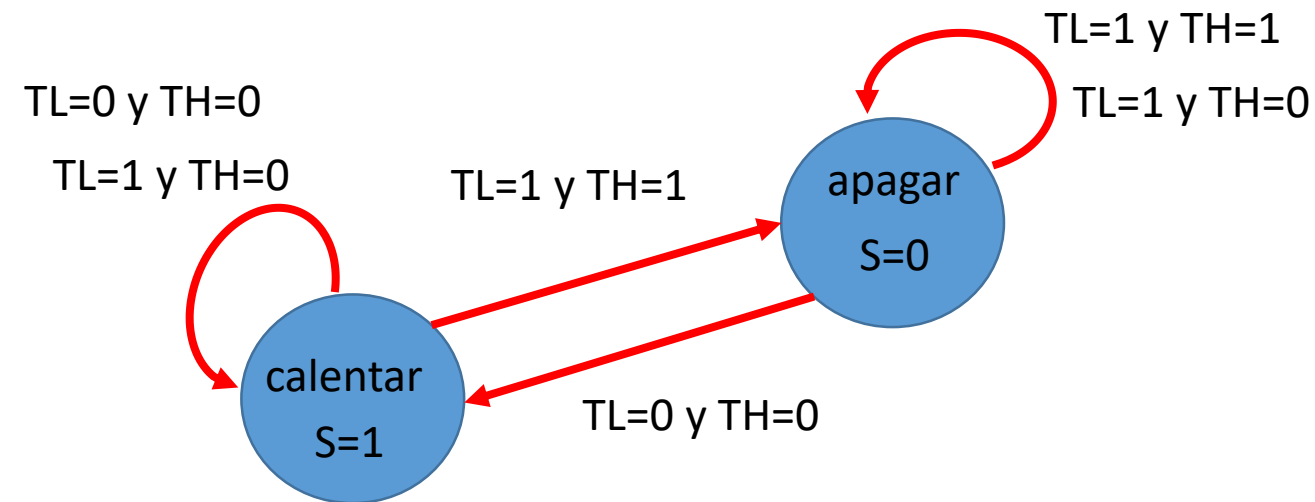


5.5 Ciclo de Histéresis con MEF

- Resultados obtenidos en el debug.

Expression	Type	Value
Temp	float	59.2999992
TH	_Bool	false
TL	_Bool	false
estado	estados_sistema	CALENTAR

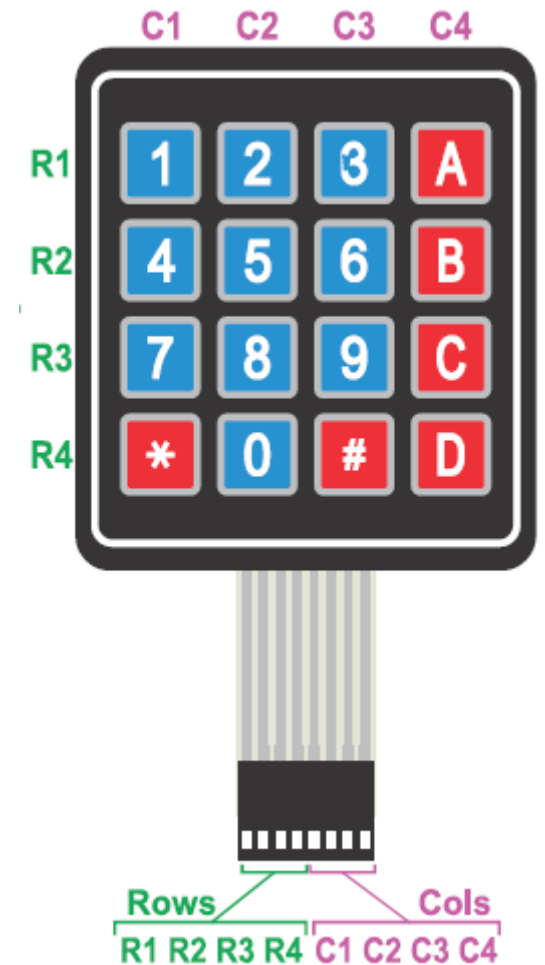
Expression	Type	Value
Temp	float	80.8000031
TH	_Bool	false
TL	_Bool	true
estado	estados_sistema	CALENTAR



Expression	Type	Value
Temp	float	96.0999985
TH	_Bool	true
TL	_Bool	true
estado	estados_sistema	APAGAR

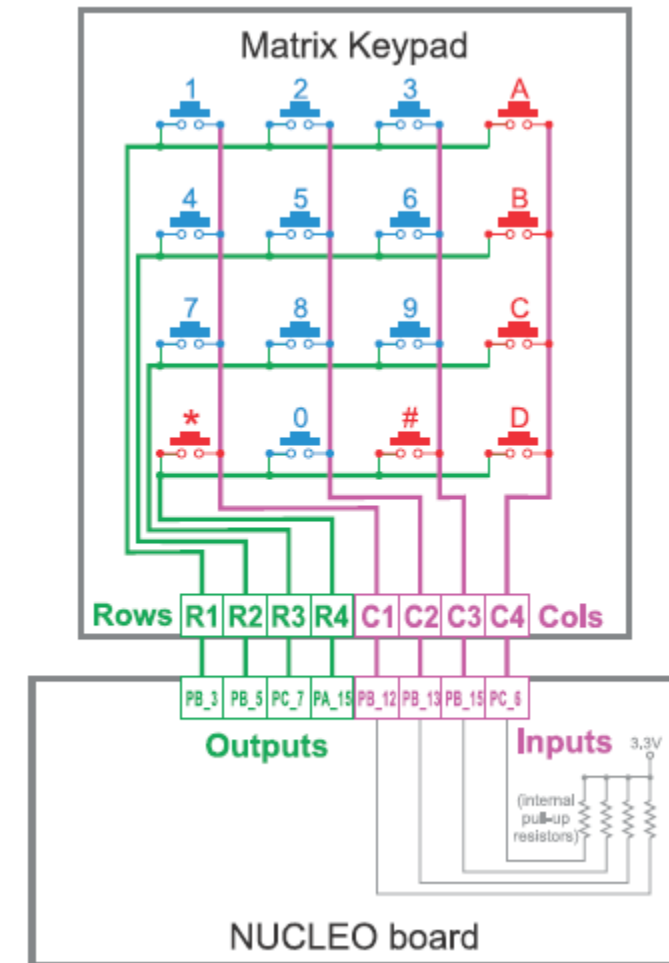
5.6 Teclado Matricial.

- Teclado matricial: dispositivo de entrada que organiza una serie de botones en una cuadrícula de filas y columnas.
- En lugar de conectar cada botón individualmente, los botones se disponen en una matriz.
- Por ejemplo, en un teclado de 4x4.



5.6 Teclado Matricial.

- ¿Cómo funciona físicamente?
- El teclado funciona mediante un método de barrido o escaneo:
- Estado inicial: Todas las filas y columnas están configuradas como entradas con resistencias pull-up o pull-down.



5.6 Teclado Matricial.

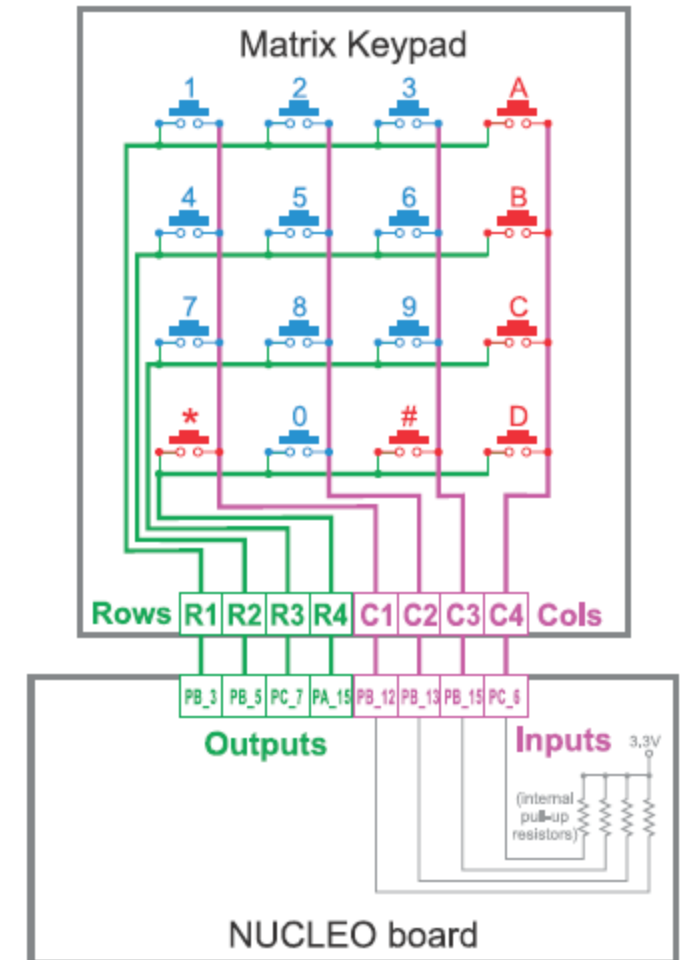
- El escaneo:

```
1 char matrixKeypadScan()
2 {
3     int row = 0;
4     int col = 0;
5     int i = 0;
6
7     for( row=0; row<KEYPAD_NUMBER_OF_ROWS; row++ ) {
8
9         for( i=0; i<KEYPAD_NUMBER_OF_ROWS; i++ ) {
10             keypadRowPins[i] = ON;
11         }
12
13         keypadRowPins[row] = OFF;
14
15         for( col=0; col<KEYPAD_NUMBER_OF_COLS; col++ ) {
16             if( keypadColPins[col] == OFF ) {
17                 return matrixKeypadIndexToCharArray[row*KEYPAD_NUMBER_OF_ROWS + col];
18             }
19         }
20     }
21     return '\0';
22 }
```

- El microcontrolador activa una sola fila a la vez y lee el estado de todas las columnas.
- Si una columna cambia de estado, el microcontrolador sabe exactamente qué tecla fue presionada.

5.6 Teclado Matricial.

- Próxima clase, aplicaciones con teclado matricial.



Bibliografia.

Lutenberg A., Gomez P. and Pernia E., 2022. A Beginner's Guide to Designing Embedded System Applications on Arm Cortex-M Microcontrollers, Arm Education Media.

Ortiz J.P., Malagón P. J., Cárdenas R. and Gómez J.J., 2025. Fundamentos teóricos de sistemas basados en microcontrolador STM32, Sistemas Digitales II Sistemas Electrónicos, Universidad Politécnica de Madrid.

Cem Ünsalan C., Yücel M. E. and Gürhan H. D., 2018. Digital Signal Processing using Arm Cortex-M based Microcontrollers-Theory and Practice, Arm Education Media.

YIU J., 2019. System-on-Chip Design with Arm® Cortex® -M Processors, Arm Education Media